Xuerou Cai
Shawn Meng
Jacklyn Trinh
Justin Trinh
CPE 301 – Embedded System Design (Spring 2025)

Final Project: Evaporation Cooling System

1. Project Description

This project implements a functional evaporative cooling system (swamp cooler) using an Arduino Mega 2560 and sensors from the Elegoo kit. The system monitors temperature, humidity, and water level, and controls airflow through a fan and an adjustable vent. It operates through four states, which are DISABLED, IDLE, RUNNING, and ERROR, each are a different mode of operation for the cooler.

During regular operation, the DHT11 sensor reads temperature and humidity, and the results are shown on a 16×2 LCD. When the temperature rises above a set threshold, the fan turns on to provide cooling, and turns off once conditions drop back down. The vent direction can also be changed at any time through a 28BYJ-48 stepper motor, which is controlled by a simple user-adjustable potentiometer. The water level sensor continuously checks the reservoir. If the water level becomes too low, the cooler immediately enters the ERROR state, turns the fan off, and displays a warning on the LCD until the reservoir is refilled and Reset is pressed. The Start button is handled through an interrupt so the system can wake up instantly, while Stop and Reset help move between states as needed.

All core control was done at the register level. ADC readings, digital input handling, and UART communication were implemented through direct register access instead of using built-in Arduino functions like analogRead, digitalWrite, or Serial print statements. Only the required libraries were used for the LCD, DHT11, stepper motor, and real-time clock. The clock logs every state change, fan event, and vent movement, and sends those timestamps to the computer through USB.

2. Component Details

The evaporative cooling system uses several electrical and mechanical components working together to sense the environment, make decisions, and move air. Below is a description of every component used in the final design and what role it plays in the system.

- Arduino Mega 2560

The Arduino Mega is the main controller of the cooler. It handles all sensor inputs, runs the finite-state machine, controls the fan and vent motor, and manages LCD output and logging. All

core logic such as ADC sampling, reading buttons, and UART communication was done using register-level programming rather than the built-in Arduino functions.

- Temperature and Humidity Sensor

The sensor measures the current temperature and humidity of the surrounding environment. The system uses its temperature readings to decide when to turn the fan on or off, and both values are displayed on the LCD for the user to monitor conditions.

- Water Level Sensor

This sensor checks whether the reservoir has enough water for safe operation. If the water becomes too low, the cooler immediately switches to the ERROR state, disables the fan, and instructs the user to refill the reservoir. The system only leaves ERROR once the reservoir is full and Reset is pressed.

- 16×2 LCD Display

The LCD displays temperature, humidity, and the current state of the system. It also shows warnings during low-water conditions. All updates occur once per minute to follow project timing rules and avoid unnecessary screen refreshes.

- DC Fan with External Motor Driver

The fan simulates air circulation. It turns ON when the DHT11 temperature crosses a threshold and turns OFF once it cools back down. The fan is powered through an external driver for safety because it cannot be connected directly to the Arduino output pins.

- Stepper Motor + ULN2003 Driver

This motor controls the cooler's vent direction. Movement is controlled by a potentiometer, allowing users to adjust airflow. The ULN2003 driver safely handles current to the motor and receives commands from the Arduino.

- Potentiometer

The potentiometer acts as the input for adjusting vent direction. Turning the knob changes the motor position, and the RTC logs significant movement changes.

- Real-Time Clock (DS1307)

The RTC keeps accurate time and logs all important actions, such as fan start and stop events, state transitions, and vent angle changes. These logs are sent over USB using low-level UART.

- Start, Stop, and Reset Buttons

The Start button is connected to an interrupt, allowing the system to wake up immediately. Stop forces the system into DISABLED, and Reset exits the ERROR state only when water has been restored.

- Indicator LEDs (Yellow, Green, Blue, Red)

Each LED indicates a different state: DISABLED (Yellow), IDLE (Green), RUNNING (Blue), and ERROR (Red). These give immediate visual feedback even without the LCD.

3. System Overview

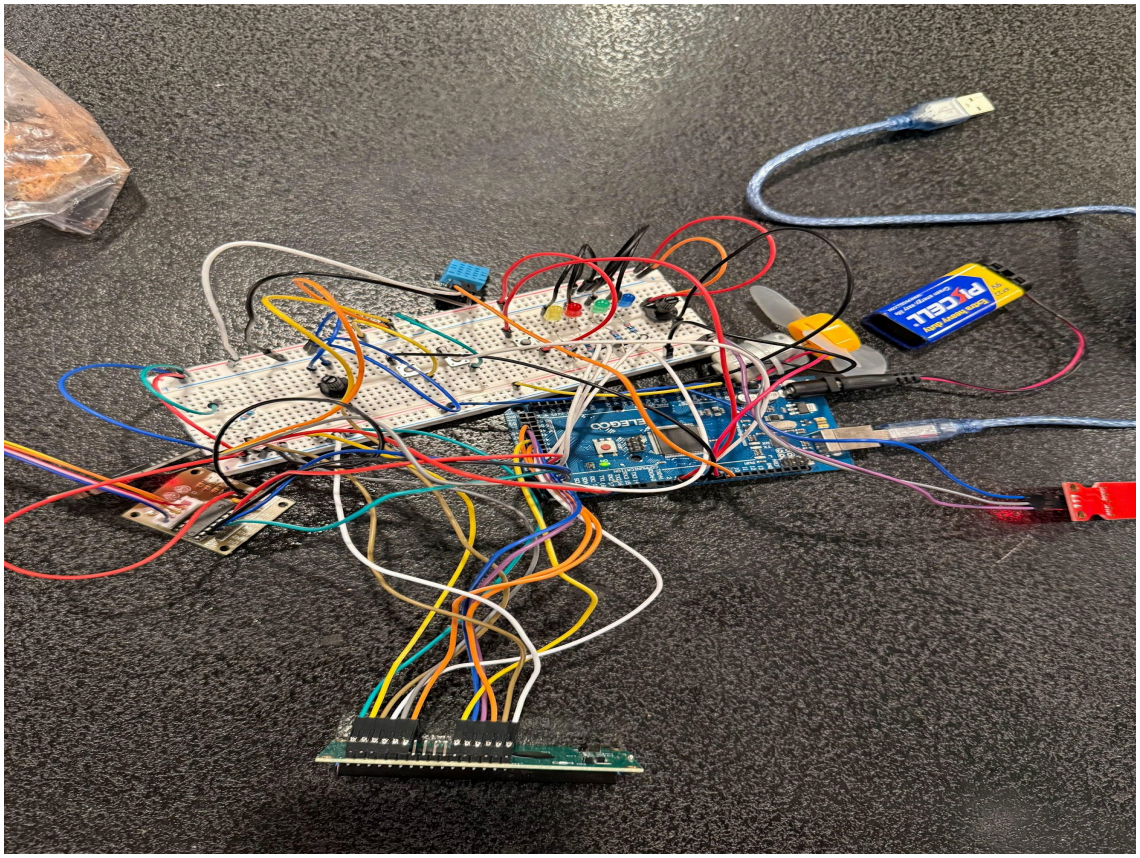
The evaporative cooler operates as a state-based embedded system focused on safe temperature control and reliable airflow. The design continuously monitors its environment and only allows cooling when there is enough water in the reservoir and when the temperature reaches a point where cooling is needed. Because evaporative cooling relies on water to be effective, the software disables the fan whenever water is too low to protect both the system and the user.

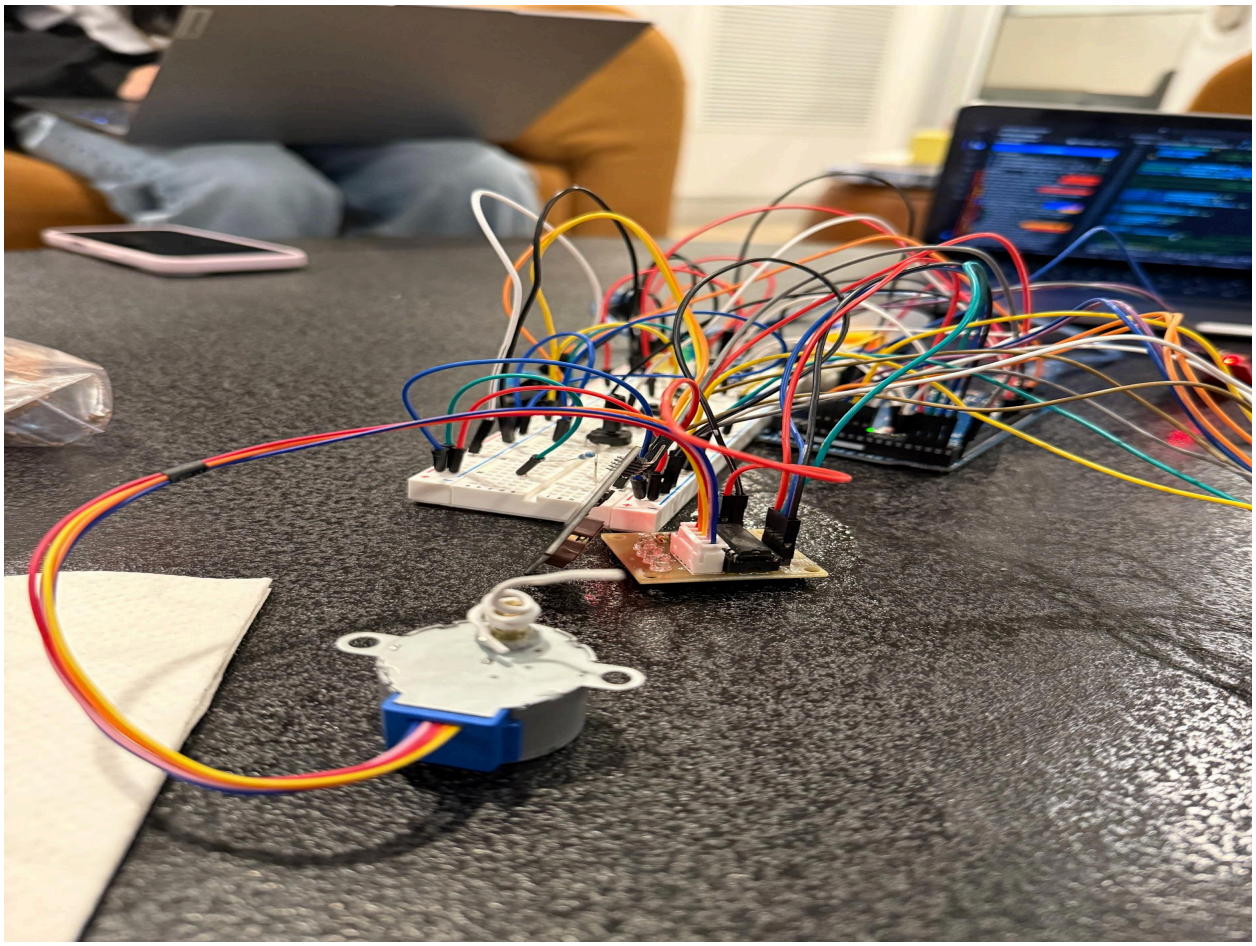
The entire design is powered through an external 5V supply from the Arduino Mega, with the fan powered separately through a driver to avoid drawing motor current directly from the board. All decision-making, sensor reading, and communication are handled through register-level control to meet course requirements. LCD updates are limited to once per minute to reduce screen wear and avoid unnecessary bus traffic. The stepper motor and real-time clock communicate through their own drivers and libraries, while core operations such as ADC input, GPIO control, and UART logging are performed without built-in Arduino functions.

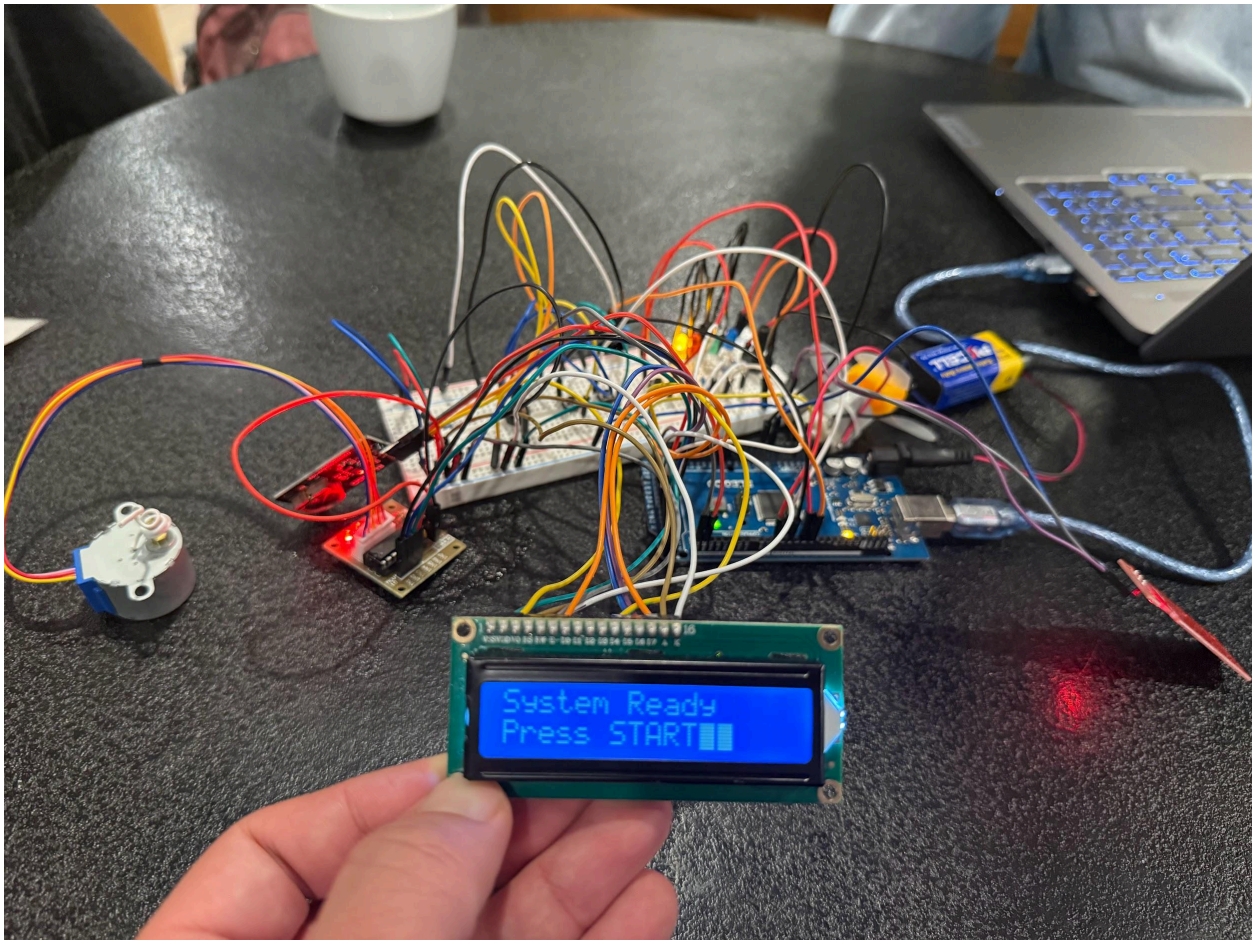
System Constraints and Behavior

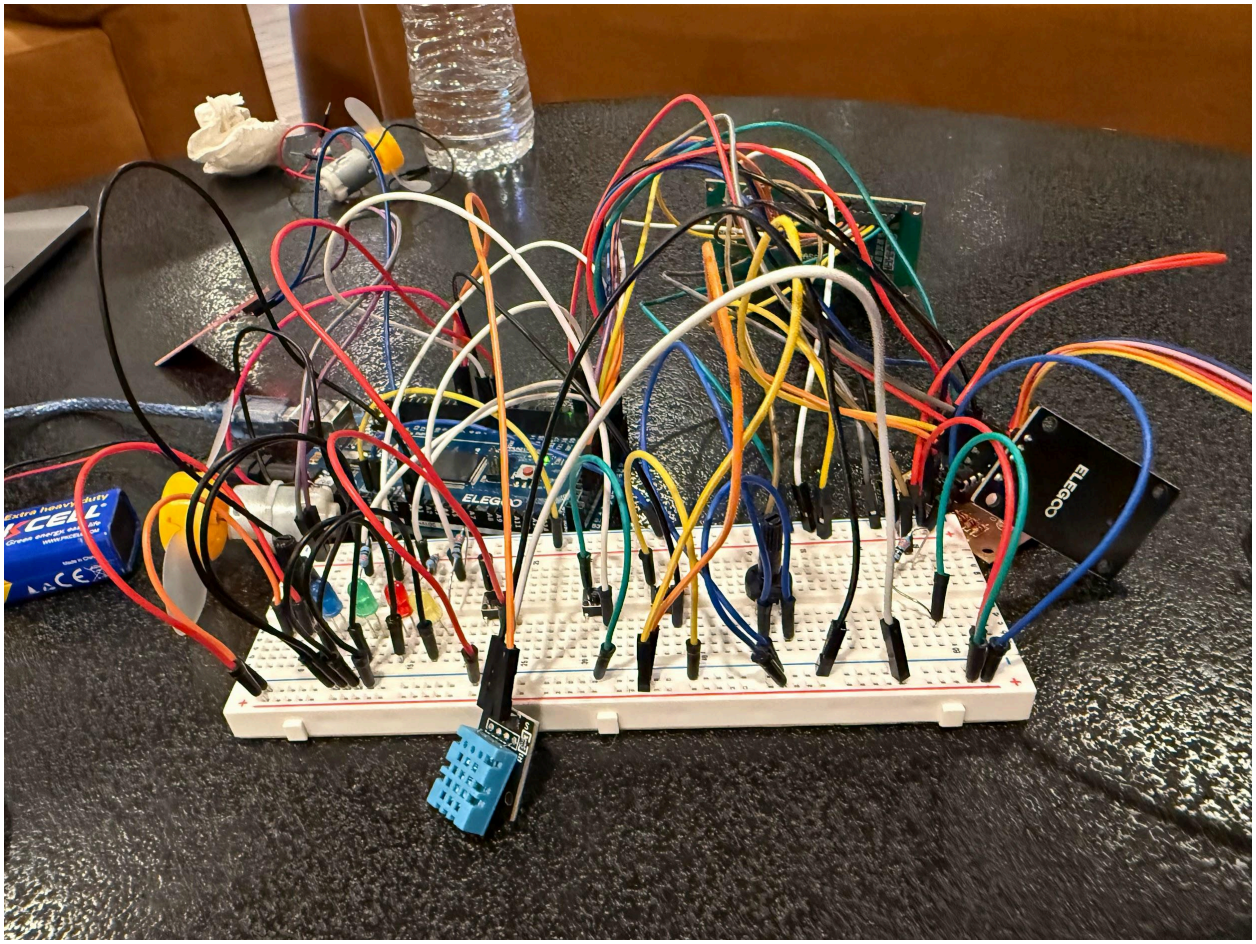
- The fan only runs when the DHT11 temperature rises above a chosen threshold and stops when cooling lowers it below a second threshold.
- Water level must be above the required minimum; otherwise the system enters ERROR regardless of temperature.
- The fan cannot be powered directly from the Arduino and must use an external driver due to current limitations.
- LCD updates are restricted to once per minute using millis() timing.
- Only allowed component libraries may be used; all other I/O communication must be done through direct register access.
- Vent movement is disabled in DISABLED and ERROR states to prevent unnecessary adjustments or unsafe operation.
- RTC timekeeping ensures that every motor action, state change, and event is logged with accurate timestamps over USB.

4. Circuit Images


PICEL




System Ready
Press START



5. Schematic Diagram

6. System Demonstration

7. Submission Links

GitHub Link: https://github.com/justintrinh10/CPE301-Final
Video Link: